

AnonyFeedback Pro

v2.1.0

Système de retours anonymes pour l'amélioration
continue des services publics et privés

Documentation technique et conceptuelle
Application mobile Flutter + Plateforme web Next.js + Supabase

Contexte : Madagascar et Afrique

	Ta
	4 1. In
1.1 Objectifs du document	4
1.2 Public vise	4
	5 2. Ar
2.1 Besoins des utilisateurs (citoyens)	5
2.2 Besoins des responsables (admins)	5
2.3 Besoins fonctionnels transverses	5
	6 3. Ar
3.1 Contrainte d'anonymat maximal	6
3.2 Contrainte technique : low-end et faible bande passante	6
3.3 Contrainte de fiabilite des donnees	6
3.4 Contraintes de securite et de roles	6
	7 4. Ar
	8 5. Po
5.1 Une seule application pour le citoyen	8
5.2 Effet de reseau et masse critique	8
5.3 Mutualisation technique et reduction des couts	8
5.4 Comparaison inter-secteurs et pilotage public	8
5.5 Adaptabilite par templates	8
	9 6. Co
6.1 Diagramme de cas d'utilisation	9
6.2 Diagramme de classes (modele de donnees)	10
6.3 Diagramme de sequence (soumission d'un feedback)	10
	11 7. Co

1. Introduction

AnonyFeedback Pro est une solution complete de collecte et de gestion de retours (feedbacks) totalement anonymes. Elle permet a tout citoyen de donner son avis sur n'importe quel service - ecole, hopital, clinique, boutique, restaurant, hotel, administration publique, transport - et aux responsables de suivre, analyser et agir rapidement grace a une plateforme web professionnelle.

La version 2.0.0 marque une evolution majeure : un formulaire intelligent avec validation en temps reel et logique conditionnelle, un suivi du statut des feedbacks (recu, en cours, resolu), des alertes intelligentes en cas de probleme grave, une plateforme d'administration web dediee (Next.js 15), ainsi que de nombreuses fonctionnalites utiles : conversations anonymes, heatmap des problemes, mode express, statistiques publiques anonymisees.

1.1 Objectifs du document

Ce document presente l'analyse des besoins et des contraintes, justifie les choix technologiques et conceptuels, decrit la conception UML du systeme et conclut sur la valeur apportee par la solution.

1.2 Public vise

Equipe de developpement, responsables d'etablissements, decideurs publics, ainsi que toute partie prenante interessee par l'amelioration continue des services par la voix citoyenne.

2. Analyse des besoins

2.1 Besoins des utilisateurs (citoyens)

- Soumettre un feedback totalement anonyme, sans créer de compte.
- Identifier l'établissement concerné : nom obligatoire, adresse facultative.
- Noter un service (étoiles, échelle, smileys) - note facultative.
- Choisir un type de problème parmi des propositions, toujours disponibles.
- Joindre des photos ou une courte vidéo, et la localisation GPS si utile.
- Accéder rapidement à un établissement via un QR code.
- Suivre l'avancement de son feedback et dialoguer de manière anonyme.
- Utiliser l'application même sans connexion (offline-first).
- Choisir sa langue : malgache, français ou anglais.

2.2 Besoins des responsables (admins)

- Visualiser et filtrer tous les feedbacks par secteur, note, statut, date.
- Être alerte immédiatement en cas de feedback critique.
- Moderer, prioriser, changer le statut et publier des actions correctives.
- Analyser les tendances : positifs/négatifs, évolution, comparaisons.
- Exporter les données (CSV, Excel, PDF) et garder un historique d'audit.
- Gérer plusieurs administrateurs avec des rôles distincts.

2.3 Besoins fonctionnels transverses

Validation et contraintes robustes en temps réel (mobile et web), gestion intelligente de la logique des formulaires, transparence via des statistiques publiques anonymisées, et notifications push pertinentes.

3. Analyse des contraintes

3.1 Contrainte d'anonymat maximal

Aucune donnée permettant d'identifier l'utilisateur ne doit être stockée. La table des feedbacks ne contient aucun identifiant utilisateur : le suivi se fait via un code anonyme aléatoire connu du seul auteur. Les règles de sécurité au niveau des lignes (RLS) garantissent qu'un utilisateur ne peut jamais lire les feedbacks d'autrui.

3.2 Contrainte technique : low-end et faible bande passante

La cible inclut des téléphones Android d'entrée de gamme et des connexions instables. D'où : compression agressive des images, cartes OpenStreetMap légères, interface sobre, et surtout une architecture offline-first ou tout fonctionne hors ligne puis se synchronise automatiquement.

3.3 Contrainte de fiabilité des données

Les règles métier sont appliquées à deux niveaux : côté client (validation temps réel dans le formulaire) et côté serveur (triggers PostgreSQL). Dans la version 2.1, le seul champ obligatoire est le nom de l'établissement, afin de toujours rattacher un retour à un lieu identifiable ; la note et les détails restent facultatifs pour ne pas freiner la participation. Le bouton d'envoi demeure actif, et le type de problème est toujours proposé (en style neutre). La cohérence est garantie de bout en bout.

3.4 Contraintes de sécurité et de rôles

Authentification anonyme sur mobile, mais email/mot de passe pour les admins web, avec gestion de rôles (Super Admin, Admin de domaine, Modérateur) et journal d'audit de toutes les actions sensibles.

4. Arguments du choix des technologies

Flutter (mobile)	Un seul code pour Android/iOS, performances natives, riche ecosysteme, ideal pour une UI moderne et fluide sur appareils modestes.
Supabase (backend)	PostgreSQL managé + Auth + Storage + Realtime + Edge Functions. La securite par RLS est parfaite pour garantir l'anonymat. Open-source, evite le verrouillage propriétaire.
Isar (local)	Base locale rapide et typee pour le mode offline-first ; lecture/ecriture instantanees sans connexion, synchronisation differee.
Riverpod	Gestion d'etat moderne, testable, en couches ; rend le changement de langue et de theme instantane.
Next.js 15 (web)	App Router + rendu serveur pour un tableau de bord rapide et securise ; TypeScript pour la fiabilite, Tailwind pour une UI soignee, recharts pour des graphiques interactifs.
flutter_map + OSM	Cartographie libre, sans cle API ni cout, et legere en bande passante - adaptee au contexte local.

Ces choix forment une pile coherente, moderne, economique et adaptee aux contraintes de terrain (cout, connectivite, materiel).

5. Pourquoi couvrir tous les domaines plutôt qu'un seul

Un choix structurant de la solution est de couvrir l'ensemble des secteurs (santé, éducation, commerce, administration, restauration/hôtellerie, transport) plutôt que de se limiter à un seul domaine. Les raisons :

5.1 Une seule application pour le citoyen

Le citoyen utilise une multitude de services au quotidien. Une application unique, multi-domaines, évite d'installer dix applications différentes et augmente fortement l'adoption et la fréquence d'usage.

5.2 Effet de réseau et masse critique

Regrouper tous les secteurs crée une masse critique d'utilisateurs et de données. Plus il y a de retours, plus les statistiques sont fiables et plus la plateforme devient utile pour tous - un cercle vertueux qu'un produit mono-domaine atteint difficilement.

5.3 Mutualisation technique et réduction des coûts

L'infrastructure (anonymat, offline, synchronisation, modération, cartes, exports) est commune à tous les secteurs. La mutualiser réduit fortement le coût de développement et de maintenance, tout en assurant une expérience homogène.

5.4 Comparaison inter-secteurs et pilotage public

Couvrir tous les domaines permet des analyses transversales (par exemple comparer la satisfaction entre hôpitaux et administrations d'une même région) - une valeur stratégique pour les décideurs publics, impossible avec une approche cloisonnée.

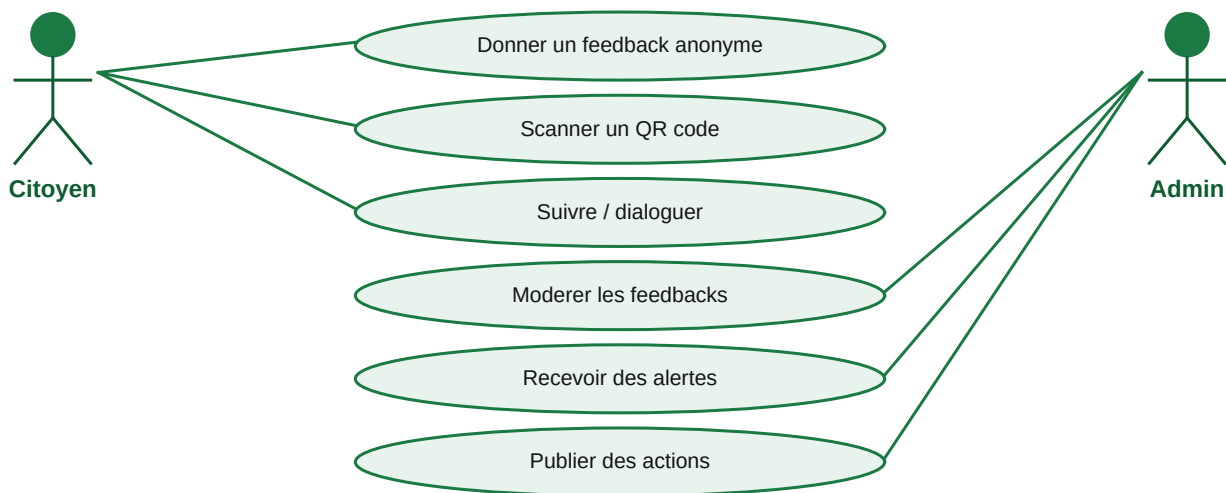
5.5 Adaptabilité par templates

La généralité ne nuit pas à la pertinence : des templates de questionnaires spécifiques par secteur, et un formulaire à logique conditionnelle, adaptent l'expérience à chaque domaine tout en gardant un socle commun.

6. Conception UML

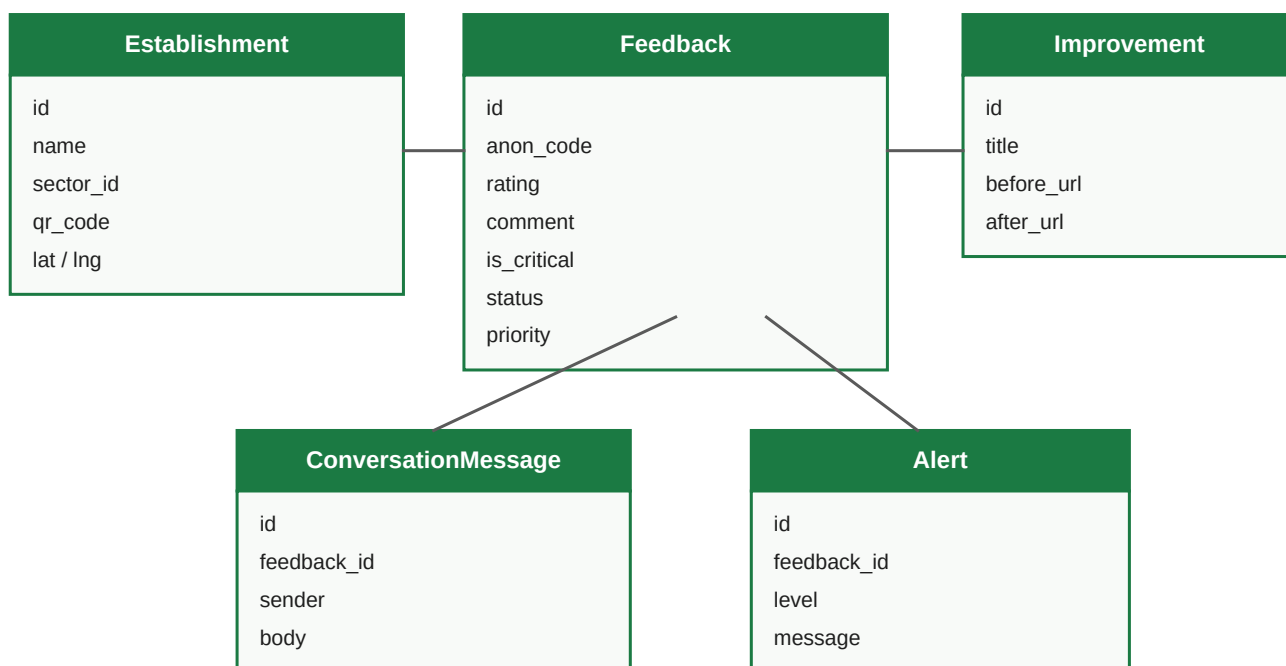
6.1 Diagramme de cas d'utilisation

Acteurs principaux et leurs interactions avec le systeme :



Le citoyen donne des feedbacks, scanne des QR codes et suit/dialogue de facon anonyme. L'admin modere, recoit les alertes et publie les actions correctives.

6.2 Diagramme de classes (modele de donnees)



Le feedback est l'entité centrale (sans aucun lien d'identité). Il se rattache à un établissement, peut générer des messages de conversation, des alertes, et donner lieu à des améliorations publiées.

6.3 Diagramme de sequence (soumission d'un feedback)

- 1. L'utilisateur saisit le nom de l'établissement (obligatoire) et remplit le formulaire (validation temps réel, note facultative).
- 2. À l'envoi, le feedback est écrit immédiatement dans Isar (local).
- 3. Si en ligne : upload des médias compressés vers Supabase Storage.
- 4. Insertion de la ligne dans PostgreSQL (RLS : insertion anonyme).
- 5. Les triggers valident, créent une alerte si critique, indexent (FTS).
- 6. L'admin voit le feedback et l'alerte en quasi temps réel (Realtime).

7. Conclusion

AnonyFeedback Pro v2.0.0 répond à un besoin concret : donner une voix anonyme et sûre aux citoyens, et des outils d'action efficaces aux responsables, dans un contexte de connectivité et de matériel contraints.

Les choix d'architecture - anonymat par conception, offline-first, règles métier appliquées à la fois côté client et serveur, pile technologique moderne et économique - garantissent fiabilité, sécurité et adoption. L'approche multi-domaines maximise l'impact social et la valeur des données.

La solution est conçue pour évoluer : conversations temps réel, intelligence artificielle d'aide à la décision, heatmap des problèmes et transparence publique constituent une feuille de route claire vers un outil de référence pour l'amélioration continue des services.

AnonyFeedback Pro v2.0.0 - Donner de la voix, améliorer les services.